AI RESUME BUILDER

Complete Technical & Interview Documentation

Author / Developer:	Likith Naidu
Target Role:	Java Full-Stack Developer / Backend Developer
Backend Stack:	Java 17, Spring Boot 4.0.6, Spring Data JPA, Hibernate, MySQL
Frontend Stack:	React 19, Vite 8, Axios, HTML2Canvas, jsPDF, docx
AI Integration:	Gemini API (via RestTemplate & Spring Service)
Date of Document:	September 2026
Codebase Location:	C:\Users\likhi\Downloads\resume-builder\resume-builder

Note to Interviewer / Technical Evaluator: *This document contains a comprehensive 25-section deep dive into the architecture, design patterns, database schema, REST APIs, AI integration, security safeguards, performance attributes, and scenario-based interview questions for the AI Resume Builder project, derived directly from the audited codebase.*

TABLE OF CONTENTS

SECTION 1 — PROJECT OVERVIEW

1.1 Project Name & High-Level Concept

AI Resume Builder is a full-stack, web-based application designed to streamline the creation, management, storage, formatting, and AI-enhancement of professional software engineering resumes.

1.2 Problem Statement

Job seekers—particularly freshers and junior developers—frequently struggle with two critical aspects of job applications:

- **Inconsistent Formatting:** Creating clean, ATS-compliant, single-page professional layouts without complex desktop publishing tools.
- **Weak Professional Summaries:** Crafting concise, impact-driven objective statements that highlight technical competencies without fluff or grammatical errors.

1.3 Target Audience & Core Value Proposition

The system targets software engineering students, fresh graduates, and active job seekers. It eliminates manual formatting friction by rendering a real-time, ATS-friendly preview alongside input forms and provides one-click AI summary generation leveraging large language models.

1.4 Technical vs Simple English Explanation

Technical Explanation: The application follows a decoupled Client-Server architecture. The frontend is built with React 19 and Vite, interacting via Axios with a Spring Boot 4.0.6 RESTful backend service. Persistent state is stored in a MySQL relational database via Spring Data JPA (Hibernate). The backend integrates with the Gemini LLM REST API via Spring's RestTemplate to perform automated prompt engineering and return tailored candidate summaries.

Simple Interview Explanation: 'I built an AI Resume Builder that allows users to fill out their details in a form, see a live preview of an ATS-compliant resume, save their profiles into a MySQL database, and click a button to generate a 3-line professional AI summary powered by an LLM.'

SECTION 2 — ONE-MINUTE INTERVIEW EXPLANATION

Interview Prompt: *'Tell me about your project.'*

*"I developed an **AI-Powered Resume Builder** using **Spring Boot, React, and MySQL**. The main objective of this project is to help job seekers quickly generate professional, ATS-friendly resumes with AI-crafted summaries.*

*On the frontend, I used **React with Vite** to build a dynamic single-page interface where users enter candidate details like education, skills, projects, and work experience, while viewing a **real-time preview**. Users can export their formatted resume directly as a PDF or editable DOCX document.*

*On the backend, I built a **RESTful service using Spring Boot 4** with layered architecture—Controllers, Services, and Repositories. Candidate records are saved in **MySQL using Spring Data JPA** with Bean Validation to ensure data integrity.*

*A key feature is the **AI Summary Generator**. When a user clicks 'Generate AI Summary', the backend formats candidate metadata and communicates with the **Gemini API** via RestTemplate. The generated summary is automatically sanitized and persisted into the database. I also implemented global exception handling, CORS configuration, and OpenAPI documentation with Swagger."*

SECTION 3 — COMPLETE FEATURE LIST

3.1 Implemented Features (Audited Codebase)

- **Full Resume CRUD Operations:** Users can Create, Read All, Read by ID, Update, and Delete candidate resume records via REST APIs backed by MySQL persistence.
- **Real-Time ATS Preview:** React state synchronized preview pane rendering candidate data immediately into a structured, single-page ATS-compliant paper layout.
- **AI Professional Summary Generation:** Spring Boot service (`AiService`) constructs tailored prompts with candidate skills and experience, sending HTTP POST requests to Gemini API and returning 3-line summaries.
- **Automated Summary Persistence:** Executing `PUT /api/resumes/{id}/generate-summary` calls the AI service, validates the output, and updates the database record in MySQL in a single transaction.
- **Multi-Format Client Export:** Export resume preview to high-resolution PDF (`html2canvas` + `jsPDF`) and native editable Word document (`docx` + `file-saver`).
- **DTO Bean Validation & Exception Handling:** Spring `@Valid`, `@NotBlank`, `@Size` annotations protect endpoints, returning structured HTTP 400 Bad Request error maps via `@ControllerAdvice`.
- **Interactive OpenAPI / Swagger Docs:** Springdoc OpenAPI 2.8 integration serving interactive API documentation at `/swagger-ui.html`.

3.2 Unconfirmed / Future Features (Explicit Roadmap Demarcation)

Audit Notice: *The following features are **NOT** present in the current audited codebase and are explicitly categorized as future enhancements:*

1. ***Job Fit PDF Upload & Parsing:*** *Processing raw PDF resume/job description files via Apache PDFBox.*
2. ***Weighted Match Scoring Engine:*** *Deterministic scoring algorithm (Skills 40%, Keywords 25%, etc.).*
3. ***Authentication & Authorization:*** *JWT / Spring Security login pipeline.*

SECTION 4 — TECH STACK SPECIFICATION MATRIX

Technology	Version	Role / Layer	Why Chosen	Alternative & Trade-off
Java	17 (LTS)	Backend Language	Strong typing, high performance, robust ecosystem.	Python (slower, dynamic typing).
Spring Boot	4.0.6	Backend Framework	Rapid REST creation, dependency injection, auto-config.	Node.js/Express (callback complexity).
Spring Data JPA	4.0.6	ORM / Data Access	Eliminates boilerplate SQL, automatic schema mapping.	Raw JDBC / MyBatis (verbose code).
MySQL	8.0+	Relational Database	ACID compliance, structured schema, wide usage.	MongoDB (lacks relational constraints).
React	19.2.6	Frontend UI Library	Component reusability, Virtual DOM, fast UI state sync.	Angular (heavy, steep learning curve).
Vite	8.0.12	Frontend Build Tool	Lightning fast ESM dev server, fast production builds.	Create React App / Webpack (slow).
Axios	1.18.1	HTTP Client	Automatic JSON transformation, interceptors, error handling.	Native Fetch (requires manual JSON parsing).
Gemini API	v1beta	AI Summary LLM	Fast inference latency, strong zero-shot prompt adherence.	OpenAI GPT-4 (higher cost/latency).
jsPDF / html 2canvas	4.2 / 1.4	PDF Rendering	Client-side DOM-to-canvas rendering, zero server load.	Server-side iText/wkhtmltopdf (heavy).
docx / file-saver	9.7 / 2.0	DOCX Generation	Client-side structured Word doc creation and download.	Apache POI on backend (high memory).

SECTION 5 — SYSTEM ARCHITECTURE

5.1 Decoupled Tier Architecture

The system follows a standard 3-tier enterprise pattern (Presentation, Business Logic, Data Storage) plus an External Integration tier.

Layer	Component	Responsibility
1. Presentation Tier	React 19 SPA (Vite)	Captures user input, handles form state, renders ATS preview, triggers export.
2. API Transport Tier	Axios / HTTP REST	Transmits JSON payloads over HTTP port 8080 with CORS handling.
3. Controller Tier	`ResumeController` `AiController`	Validates DTOs (`@Valid`), maps REST endpoints, returns `ResponseEntity`.
4. Service Tier	`ResumeService` `AiService`	Executes business logic, prompt construction, LLM response parsing, summary update.
5. Persistence Tier	`ResumeRepository` (JPA)	Abstracts SQL queries, maps objects to MySQL database tables.
6. Database Tier	MySQL Relational DB	Stores `resumes` table records persistently.
7. AI External Tier	Gemini REST API	Processes prompts via LLM inference and returns text candidates.

SECTION 6 — PROJECT FOLDER & FILE STRUCTURE

Below is the actual audited folder layout of the unified repository:

```
C:\Users\likhi\Downloads\resume-builder\resume-builder\
■■■ .gitignore                          # Project git ignore rules
■■■ pom.xml                             # Maven dependencies & build config
■■■ mvnw / mvnw.cmd                     # Maven wrapper scripts
■■■ HELP.md                             # Spring Boot parent guide
■■■ src/
■   ■■■ main/
■   ■   ■■■ java/com/resumebuilder/resume_builder/
■   ■   ■   ■■■ ResumeBuilderApplication.java   # Main Spring Boot Entrypoint
■   ■   ■   ■■■ config/
■   ■   ■   ■   ■■■ AppConfig.java             # RestTemplate Bean Config
■   ■   ■   ■   ■■■ OpenApiConfig.java         # Swagger OpenAPI 3.0 Setup
■   ■   ■   ■■■ controller/
■   ■   ■   ■   ■■■ ResumeController.java      # Resume CRUD & Summary Endpoints
■   ■   ■   ■   ■■■ AiController.java          # Standalone AI Summary Endpoint
■   ■   ■   ■■■ service/
■   ■   ■   ■   ■■■ ResumeService.java         # Core CRUD & DB Summary Logic
■   ■   ■   ■   ■■■ AiService.java             # Gemini API REST Client & Prompting
■   ■   ■   ■■■ repository/
■   ■   ■   ■   ■■■ ResumeRepository.java      # Spring Data JPA Repository
■   ■   ■   ■■■ model/
■   ■   ■   ■   ■■■ Resume.java               # JPA Entity (Table: resumes)
■   ■   ■   ■■■ dto/
■   ■   ■   ■   ■■■ ResumeDto.java            # Validation DTO (Create/Update)
■   ■   ■   ■   ■■■ AiSummaryRequest.java     # AI Input Payload DTO
■   ■   ■   ■   ■■■ AiSummaryResponse.java    # AI Output Payload DTO
■   ■   ■   ■■■ exception/
■   ■   ■       ■■■ ResumeNotFoundException.java# Custom 404 Runtime Exception
■   ■   ■       ■■■ GlobalExceptionHandler.java # @ControllerAdvice Exception Handler
■   ■   ■■■ resources/
■   ■       ■■■ application.properties         # DB & AI Config Placeholders
■   ■■■ test/
■       ■■■ java/...                          # Spring Boot Context Tests
■■■ resume-builder-frontend/
    ■■■ package.json                    # Frontend dependencies & scripts
    ■■■ vite.config.js                  # Vite server & build options
    ■■■ index.html                      # HTML5 Root Container
    ■■■ src/
        ■■■ main.jsx                    # React DOM Root Mounting
        ■■■ App.jsx                     # Root Component & State Coordinator
        ■■■ index.css                   # App Styles & ATS Resume Paper Styling
        ■■■ api/
        ■   ■■■ resumeApi.js            # Axios Endpoints for Backend REST
        ■■■ components/
        ■   ■■■ ResumeForm.jsx          # Controlled Input Form Component
        ■   ■■■ ResumePreview.jsx       # Live ATS Resume Preview Component
        ■   ■■■ ResumeList.jsx          # Saved Resumes Card List & AI Action
        ■   ■■■ DownloadButton.jsx      # PDF & DOCX Export Trigger Buttons
        ■■■ utils/
            ■■■ downloadUtils.js        # html2canvas/jsPDF & docx Packers
```

SECTION 7 — FRONTEND DEEP DIVE (REACT 19)

7.1 State Management Architecture in App.jsx

The React application utilizes single-directional data flow with centralized state lifted up to `App.jsx`:

- `formData`: Holds the active form fields (`fullName`, `email`, `phone`, `summary`, `skills`, `education`, `experience`, `projects`, etc.). Passed down to `ResumeForm` and `ResumePreview`.

- `selectedResume`: Tracks the currently selected candidate from the list view.
- `refreshList`: A boolean toggle state used to trigger re-fetching of saved resumes in `ResumeList` upon save or summary update.

7.2 Component Responsibilities

- **`ResumeForm.jsx`:** Controlled form inputs bound to `formData`. Executes `createResume(formData)` via `resumeApi.js` on submission. Displays inline loading and error notifications.
- **`ResumePreview.jsx`:** Renders the `#resume-preview-download` DOM node mimicking an A4 paper layout. Implements helper `formatLines()` to transform multiline text into structured paragraphs or bullet points (`line.startsWith('•')`).
- **`ResumeList.jsx`:** Calls `getAllResumes()` inside `useEffect`. Maps over records to display cards with candidate details. Contains per-card `loadingId` state during `handleGenerateSummary()` execution.
- **`downloadUtils.js`:** Implements `downloadResumeAsPdf()` (converts DOM element to canvas at scale 3 and renders pages with `jsPDF`) and `downloadResumeAsDocx()` (constructs native OOXML documents with `docx` library).

SECTION 8 — BACKEND DEEP DIVE (SPRING BOOT 4)

8.1 Layered Architecture Pattern

The backend adheres strictly to the classic Spring layered pattern:

1. **Controller Layer (`@RestController`):** Handles HTTP routing, request/response headers, and input DTO validation using `@Valid`.
2. **Service Layer (`@Service`):** Encapsulates business rules, transactions, and external integration logic.
3. **Repository Layer (`@Repository`):** Extends `JpaRepository` to execute object-relational mapping without handwritten SQL.

8.2 Request Lifecycle Sequence

Client Request → `CorsFilter` → `@RestController` (`@Valid` check) → `GlobalExceptionHandler` (if invalid) → `@Service` → `JpaRepository` → MySQL Database → Response DTO → HTTP Response (200/201).

SECTION 9 — DATABASE DESIGN & JPA MODEL

9.1 Database Schema (`resumes` Table)

The database contains a primary entity mapped to the `resumes` table in MySQL:

Column Name	Data Type	Constraints	Description / Field
`id`	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique candidate record ID.
`full_name`	VARCHAR(255)	NOT NULL (via DTO)	Candidate full name.
`email`	VARCHAR(255)	NOT NULL (via DTO)	Candidate email address.
`phone`	VARCHAR(255)	NOT NULL (10 digits)	Contact phone number.
`linkedin`	VARCHAR(255)	NULLABLE	LinkedIn profile URL.
`github`	VARCHAR(255)	NULLABLE	GitHub repository URL.

`summary`	VARCHAR(1000)	@Column(length = 1000)	Professional objective summary.
`skills`	VARCHAR(1000)	@Column(length = 1000)	Technical skills list.
`education`	VARCHAR(1000)	@Column(length = 1000)	Degree and education details.
`experience`	VARCHAR(2000)	@Column(length = 2000)	Work/internship experience.
`projects`	VARCHAR(2000)	@Column(length = 2000)	Technical project details.
`technologies`	VARCHAR(255)	NULLABLE	Specific technologies used.
`libraries`	VARCHAR(255)	NULLABLE	Libraries/frameworks.
`soft_skills`	VARCHAR(255)	NULLABLE	Soft skills / interpersonal skills.
`certifications`	VARCHAR(2000)	@Column(length = 2000)	Certifications and courses.
`education_institution`	VARCHAR(255)	NULLABLE	College/University name.

SECTION 10 — REST API SPECIFICATION REFERENCE

POST /api/resumes — Create Resume (Status: 201 CREATED)

Accepts `@Valid ResumeDto` in JSON request body, persists new `Resume` entity to MySQL, returns saved entity with generated ID.

GET /api/resumes — Get All Resumes (Status: 200 OK)

Fetches all candidate resumes from MySQL via `resumeRepository.findAll()`, returning a JSON array.

GET /api/resumes/{id} — Get Resume By ID (Status: 200 OK / 404)

Fetches candidate by Long ID. Throws `ResumeNotFoundException` (404) if ID is not present in database.

PUT /api/resumes/{id} — Update Resume (Status: 200 OK / 404)

Updates existing candidate record by ID using `@Valid ResumeDto`. Throws 404 if ID missing.

DELETE /api/resumes/{id} — Delete Resume (Status: 200 OK / 404)

Removes candidate record from MySQL database by ID. Returns success string response.

PUT /api/resumes/{id}/generate-summary — Generate & Save AI Summary (Status: 200 OK / 404)

Combines candidate skills and details, calls `AiService`, validates generated text, updates `summary` field in database.

POST /api/ai/generate-summary — Standalone AI Summary (Status: 200 OK)

Accepts `AiSummaryRequest` (fullName, skills, experience, projects), calls Gemini LLM API, returns `AiSummaryResponse` without DB write.

SECTION 11 — AI INTEGRATION DEEP DIVE (GEMINI LLM)

11.1 Spring Boot `AiService.java` Architecture

The backend integrates with Gemini REST API using Spring's `RestTemplate`:

- **Configuration Injection:** Credentials and parameters are injected using `@Value` annotations:

```
@Value("${ai.api.key}") private String apiKey;
@Value("${ai.api.url}") private String apiUrl;
@Value("${ai.api.model}") private String model;
```

• Prompt Engineering Strategy (`buildPrompt()`):

The prompt strictly constrains LLM output to prevent halluncinations and unwanted markdown formatting:

> *"Generate exactly one professional resume summary in 3 lines.*
> *Do not give multiple options. Do not use headings like Option 1 or Option 2. Do not use bullet points. Do not explain anything. Return only the final resume summary paragraph.*
> *Candidate details: Name: [fullName], Skills: [combinedSkills], Experience: [experience], Projects: [projects].*
> *Important: Do not add fake experience. Keep it suitable for a fresher or junior developer."*

• Robust Extraction Logic (`extractGeminiSummary()`):

The service safely unwraps nested JSON maps (`candidates` → `content` → `parts` → `text`) with null and empty checks to prevent `NullPointerException` crashes.

SECTION 12 — JOB FIT SCORING ARCHITECTURE (FUTURE ROADMAP)

> ***Interview Answer Strategy:*** *If asked about Job Fit scoring during an interview:*
> *'In the current version, the system focuses on automated Resume CRUD management and LLM-powered summary generation. For the upcoming Job Fit Analyzer module, I designed a hybrid scoring model where the LLM parses semantic keywords, while the Spring Boot backend computes a deterministic weighted score (Skills 40%, Keyword Match 25%, Experience 15%, Education 10%, Projects 10%). This prevents the LLM from hallucinating inconsistent numerical scores.'*

SECTION 13 — AI VS BACKEND RESPONSIBILITY MATRIX

LLM (Gemini API) Responsibility	Backend (Spring Boot) Responsibility
• Natural language summary synthesis	• Request validation (`@Valid`, Bean Validation)
• Semantic understanding of technical skills	• Database persistence & transaction management
• Adapting tone for fresher/junior profiles	• RestTemplate HTTP communication & key injection
• Adhering to 3-line output constraints	• Exception handling (`HttpClientErrorException`)
• Unstructured text transformation	• CORS policy enforcement & Swagger OpenAPI docs

SECTION 14 — SECURITY, VALIDATION & SECRET PROTECTION

14.1 Protection of API Keys

- The Gemini API key is stored in external environment variables (`.env` file) and injected dynamically via `${AI_API_KEY}`.
- `.env` files are strictly added to `.gitignore` and excluded from version control.

- Source code contains placeholder references (`${AI_API_KEY}`) to prevent credential leaks.

14.2 Input Validation & Injection Prevention

- **Bean Validation:** DTOs enforce `@NotBlank` and `@Size(min=10, max=10)` constraints.
- **SQL Injection Prevention:** Hibernate ORM uses parameterized SQL queries exclusively.
- **XSS Protection:** React automatically escapes content rendered in JSX.
- **CORS Security:** Restricts cross-origin requests to trusted local client origins (`http://localhost:5173`).

SECTION 15 — ERROR HANDLING & EXCEPTION PIPELINE

The application implements centralized exception handling via `GlobalExceptionHandler` (`@ControllerAdvice`):

- **`ResumeNotFoundException`:** Catches missing record lookups and returns JSON payload `{"message": "Resume not found with id: X", "status": 404}` with HTTP status 404 NOT FOUND.
- **`MethodArgumentNotValidException`:** Intercepts `@Valid` validation failures and returns a field error map `{"fullName": "Full name is required", "phone": "Phone number must be 10 digits"}` with HTTP status 400 BAD REQUEST.
- **AI API Exception Handling:** `AiService` catches `HttpClientErrorException` / `HttpServerErrorException` (e.g. 401 Unauthorized or quota exhaustion) and returns user-friendly error messages without crashing the server.

SECTION 16 — PERFORMANCE OPTIMIZATION & LATENCY STRATEGY

- **Single External Call:** `generateAndSaveSummary()` makes exactly one external HTTP POST request per action.
- **Client-Side Document Export:** PDF and DOCX documents are generated directly in the user's browser using `html2canvas` and `jsPDF`, offloading server CPU and memory usage.
- **Minimal Bundle Overhead:** ESM imports backed by Vite build system ensure fast browser rendering times.

SECTION 17 — TECHNICAL CHALLENGES & RESOLUTION SCENARIOS

Challenge 1: Unstructured LLM Formatting Output

Problem: The LLM initially returned bullet points, markdown headings ('Option 1'), and conversational text.

Solution: Refined prompt engineering in `buildPrompt()` with explicit negative constraints ('Do not give multiple options. Do not use bullet points. Return only the final paragraph.').

Challenge 2: Database Column Truncation

Problem: Long candidate project descriptions exceeded default MySQL `VARCHAR(255)` length limits, causing `DataException`.

Solution: Added `@Column(length = 2000)` annotations to `projects`, `experience`, and `certifications` in `Resume.java` entity.

Challenge 3: UI Sync After AI Summary Generation

Problem: Generating an AI summary in `ResumeList` did not immediately reflect in the preview or main form state.

Solution: Implemented `onSelectResume(updatedResume)` callback and lifted refresh state to update React root state in `App.jsx`.

SECTION 18 — FUNDAMENTAL 'WHY' TECHNICAL QUESTIONS

Q: Why Spring Boot?

A: Spring Boot offers rapid application development, auto-configuration, built-in dependency injection, and seamless integration with Spring Data JPA and REST controllers.

Q: Why React?

A: React's component-based architecture and Virtual DOM enable real-time UI synchronization between input forms and ATS resume previews without page reloads.

Q: Why MySQL & JPA?

A: MySQL provides ACID compliance and structured relational integrity. Spring Data JPA reduces SQL boilerplate to simple repository interfaces.

Q: Why RestTemplate for AI Integration?

A: RestTemplate provides a clean, synchronous HTTP client abstraction for making REST calls to external APIs with custom headers and DTO mappings.

Q: Why Client-Side PDF Generation?

A: Generating PDFs on the client using `html2canvas` and `jsPDF` offloads memory-heavy rendering operations from the backend server.

SECTION 19 — END-TO-END EXECUTION DATA FLOWS

19.1 Flow: Create Resume Record

User submits `ResumeForm` → Axios `POST /api/resumes` → `ResumeController.createResume()` → `@Valid` check → `ResumeService.createResume()` → `ResumeRepository.save()` → Insert into MySQL `resumes` table → Return HTTP 201 CREATED → React updates list & preview state.

19.2 Flow: Generate & Save AI Summary

User clicks 'Generate AI Summary' on candidate card → Axios `PUT /api/resumes/{id}/generate-summary` → `ResumeController.generateAndSaveSummary()` → `ResumeService.generateAndSaveSummary()` → Build candidate prompt string → `AiService.generateSummary()` → `RestTemplate.exchange()` to Gemini API → Parse JSON response → Update `resume.setSummary()` → `ResumeRepository.save()` in MySQL → Return updated candidate entity → React updates UI.

SECTION 20 — INTERVIEW CROSS-QUESTIONS (CATEGORIZED)

[Category: Spring Boot & JPA] Q: How does Spring Data JPA handle queries without handwritten SQL?

A: Spring Data JPA uses proxy instances to generate queries based on method names or entity metadata defined in `JpaRepository` interfaces.

[Category: React & State] Q: Why lift state up to App.jsx?

A: Lifting state to `App.jsx` allows sibling components (`ResumeForm`, `ResumePreview`, `ResumeList`) to share and synchronize candidate data in real time.

[Category: Exception Handling] Q: What happens when validation fails in ResumeDto?

A: `@Valid` triggers `MethodArgumentNotValidException`, which `GlobalExceptionHandler` intercepts to return an HTTP 400 Bad Request response containing a field error map.

SECTION 21 — HIGH-STAKES TECHNICAL SCENARIO QUESTIONS

Scenario: What if the Gemini AI API is down or times out?

Interview Answer: The `AiService` try-catch block catches `HttpClientErrorException` / `HttpServerErrorException` and generic `Exception`, returning a fallback message ('Unable to generate summary right now. Please try again.'). The backend does NOT crash, and existing database records remain safe.

Scenario: What if candidate text input contains malicious script tags (XSS)?

Interview Answer: React automatically escapes HTML strings rendered inside JSX elements. Additionally, Spring Boot DTO validation and JPA parameterized queries prevent SQL injection.

Scenario: How would you handle 10,000 concurrent AI summary generation requests?

Interview Answer: I would introduce an asynchronous queue (e.g. RabbitMQ/Kafka) with background worker threads to decouple HTTP requests from AI API calls, avoiding server thread pool exhaustion.

SECTION 22 — FUTURE SYSTEM ENHANCEMENTS

The following roadmap items represent planned future enhancements for Version 2.0:

1. **Job Fit PDF Analyzer:** Integration of Apache PDFBox to extract resume and job description text from uploaded PDFs.
2. **Weighted Matching Engine:** Implementation of a deterministic 5-factor scoring model.
3. **JWT Spring Security:** User registration, authentication, and multi-tenant resume authorization.
4. **Redis Caching Layer:** Caching frequent LLM queries to reduce API latency and cost.

SECTION 23 — PRODUCTION DEPLOYMENT ARCHITECTURE

- **Frontend Deployment:** Build static assets using `npm run build` in Vite and deploy to Netlify / Vercel CDN.
- **Backend Deployment:** Package Spring Boot application as an executable JAR using `mvnw clean package` and deploy to AWS EC2 / Render / Docker container.
- **Database Deployment:** Provision a managed Amazon RDS for MySQL instance with multi-AZ replication.
- **Environment Variables:** Inject production database connection strings (`DB_URL`, `DB_USERNAME`, `DB_PASSWORD`) and `AI_API_KEY` at runtime via container environment properties.

SECTION 24 — TIMED INTERVIEW SCRIPT VARIATIONS

24.1 30-Second Elevator Pitch

"I built an AI Resume Builder using Spring Boot, React, and MySQL. It lets users create professional resumes with live ATS previews, export them as PDF or Word docs, and automatically generate professional candidate summaries using the Gemini AI API."

24.2 1-Minute Summary Script

"My project is an AI Resume Builder built with a Spring Boot REST backend, a React frontend, and a MySQL database. On the frontend, users fill out candidate details and see a real-time ATS preview. They can export their resume to PDF or DOCX. On the backend, I implemented a layered architecture with Spring Data JPA and Bean Validation. When a user clicks 'Generate AI Summary', Spring Boot formats candidate skills and calls the Gemini API via RestTemplate, saving the generated summary into MySQL. I also set up global exception handling and Swagger API docs."

SECTION 25 — FINAL INTERVIEW MASTER CHEAT SHEET

Key Domain	Summary Points to Remember
Project Goal	Full-stack web application for ATS resume creation and AI summary generation.
Backend Stack	Java 17, Spring Boot 4.0.6, Spring Data JPA, Hibernate, MySQL, RestTemplate.
Frontend Stack	React 19, Vite 8, Axios, HTML2Canvas, jsPDF, docx, file-saver.
AI Integration	Gemini API REST client in `AiService` returning 3-line summaries.
Database	MySQL table `resumes` mapped via JPA entity `@Table(name = "resumes")`.
Key Security	API key externalized in `.env`, DTO `@Valid` annotations, SQL parameterization.
Top 5 Interview Golden Rules	1. Explain layered architecture (Controller → Service → Repository → MySQL). 2. Emphasize that the LLM generates text while Spring Boot manages logic and DB. 3. Mention DTO validation (`@Valid`) and `@ControllerAdvice` global error handling. 4. Explain client-side PDF export offloads backend server CPU load. 5. Be confident, concise, and highlight clean code practices.